

# **Sri Lanka Institute of Information Technology**



Year 2 - Semester 1

2025

Software Engineering (SE2030)

Group ID: 2025-Y2-S1-MLB-WE4G1-03

## **URL PhishGuard: Phishing Detection through URL Analysis Final Report**

| <b>Student ID</b> | <b>Name</b>         |
|-------------------|---------------------|
| IT24100659        | Kavindi K.N.T.      |
| IT24103625        | De Alwis A.K.M.T    |
| IT24103016        | Perera.R.H.H        |
| IT24104208        | Nethmina D.M.S.K    |
| IT24103925        | Rathnayaka R.M.S.A. |
| IT24100950        | Gunaratnam C.V      |

# 1) Introduction

In the modern digital landscape, phishing attacks remain one of the most prevalent and damaging forms of cybercrime. These attacks often exploit deceptive URLs to lure users into revealing sensitive information or downloading malicious content. As internet usage surges, the need for automated, scalable, and accurate phishing detection systems becomes increasingly critical.

This project, URL PhishGuard, aims to build a robust machine learning pipeline that analyzes URL characteristics to classify them as either legitimate ("Good") or malicious ("Bad"). By leveraging structured URL data and advanced classification models, we seek to enhance cyber security defenses and reduce the risk of phishing attacks.

## 1.1) Problem Statement

Phishing websites often mimic legitimate domains and use obfuscated URLs to deceive users. Manual detection is impractical at scale, and rule-based systems struggle to adapt to evolving attack patterns. The lack of automated, intelligent URL classification systems leaves users vulnerable to phishing threats.

## 1.2) Problem Objectives

The project will create, execute, and evaluate an end-to-end machine learning pipeline to address this classification challenge.

The specific objectives are:

- To preprocess and clean the structured URL dataset to ensure quality and consistency.
- To perform Exploratory Data Analysis (EDA) to uncover patterns and anomalies in URL structure.
- To engineer meaningful features that capture URL complexity, encoding, and structural cues.
- To train and implement six classification models (Logistic Regression, Decision Tree, Random Forest, Gradient Boosting, KNN, and Naive Bayes) capable of predicting phishing URLs.
- To evaluate and compare the performance of these models using classification metrics (Accuracy, Precision, Recall, F1-score, AUC).
- To select the most suitable model for this specific dataset and detection problem.
- To perform an ethical analysis considering possible biases and fairness implications of models and data.

## 2) Dataset Description

The dataset selected to fulfill these purposes is the "Phishing Site URLs Dataset." This data consists of 549,346 structured entries representing real-world URLs labeled as either "Good" or "Bad." The dataset contains no missing values and includes a wide range of features derived from URL structure and encoding.

This data was deemed most suitable for the project due to the following:

- **Relevance:** It offers genuine, labeled URL data, which is exactly what's required for a phishing detection assignment.
- **Volume:** The large number of entries supports robust training and generalization.
- **Feature Richness:** Fields such as `url_length_category`, `hostname_length`, `url_segments`, and `count_features_std` enable the creation of meaningful features for classification.
- **Realistic Complexity:** The dataset includes a variety of URL formats and structures, presenting a realistic opportunity to develop and demonstrate a strong preprocessing and feature engineering pipeline.

## 3) Preprocessing and EDA

### 3.1) Data Cleaning

The raw dataset, while structured and labeled, required several cleaning steps to ensure consistency, reliability, and readiness for machine learning. The following sub-steps were performed during the data cleaning phase:

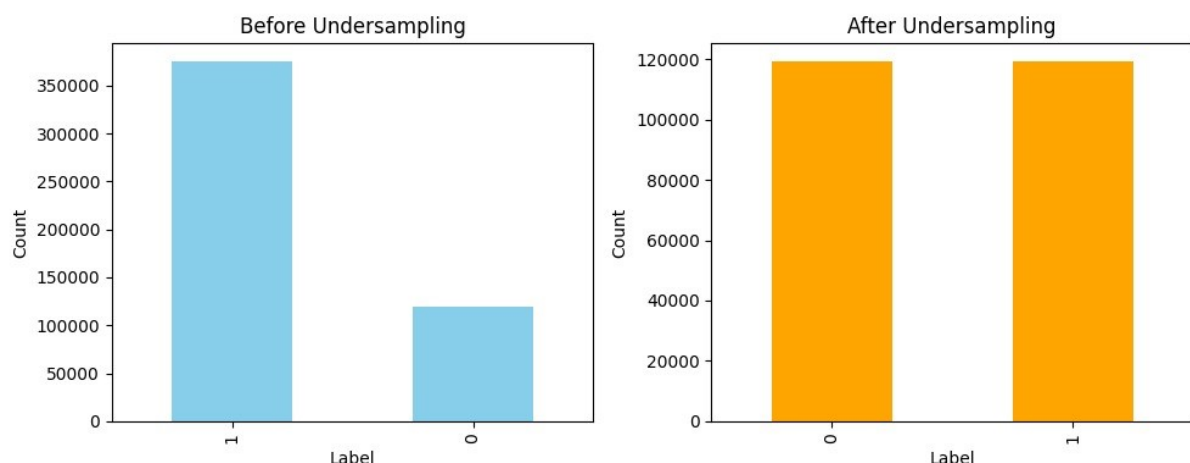
**Data Type Correction:** Columns like `hostname_length` and `url_segments` were originally stored as generic object types. These were converted to proper numeric types using `.astype(int)` to ensure smooth calculations and compatibility with machine learning models.

**Duplicate Removal:** We found and removed duplicate rows using `df.duplicated().sum()` and `df.drop_duplicates()`. This helped eliminate redundancy and ensured each URL contributed uniquely to the learning process.

**Text Standardization:** The URL column was cleaned by converting all text to lowercase and trimming extra spaces using `.str.lower()` and `.str.strip()`. This made sure similar-looking URLs were treated consistently during feature extraction.

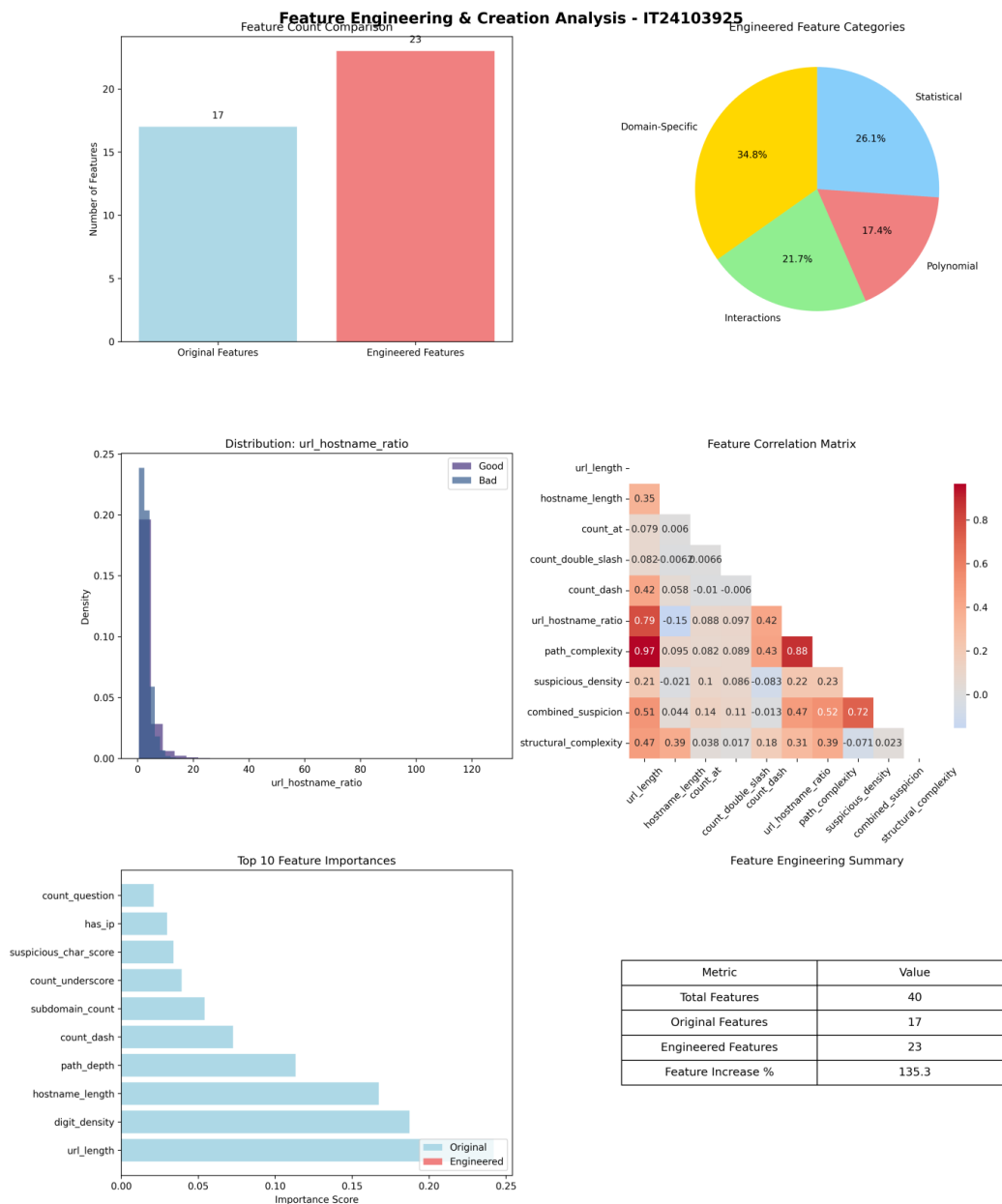
**Handling Invalid Transactions:** Some entries had unrealistic values like `url_length = 0` or `hostname_length = 1`, which don't represent valid URLs. These were filtered out to avoid skewing the feature distributions and model behavior.

**Handling Missing Data:** Although the dataset had no missing values, we ran `df.isnull().sum()` to confirm.



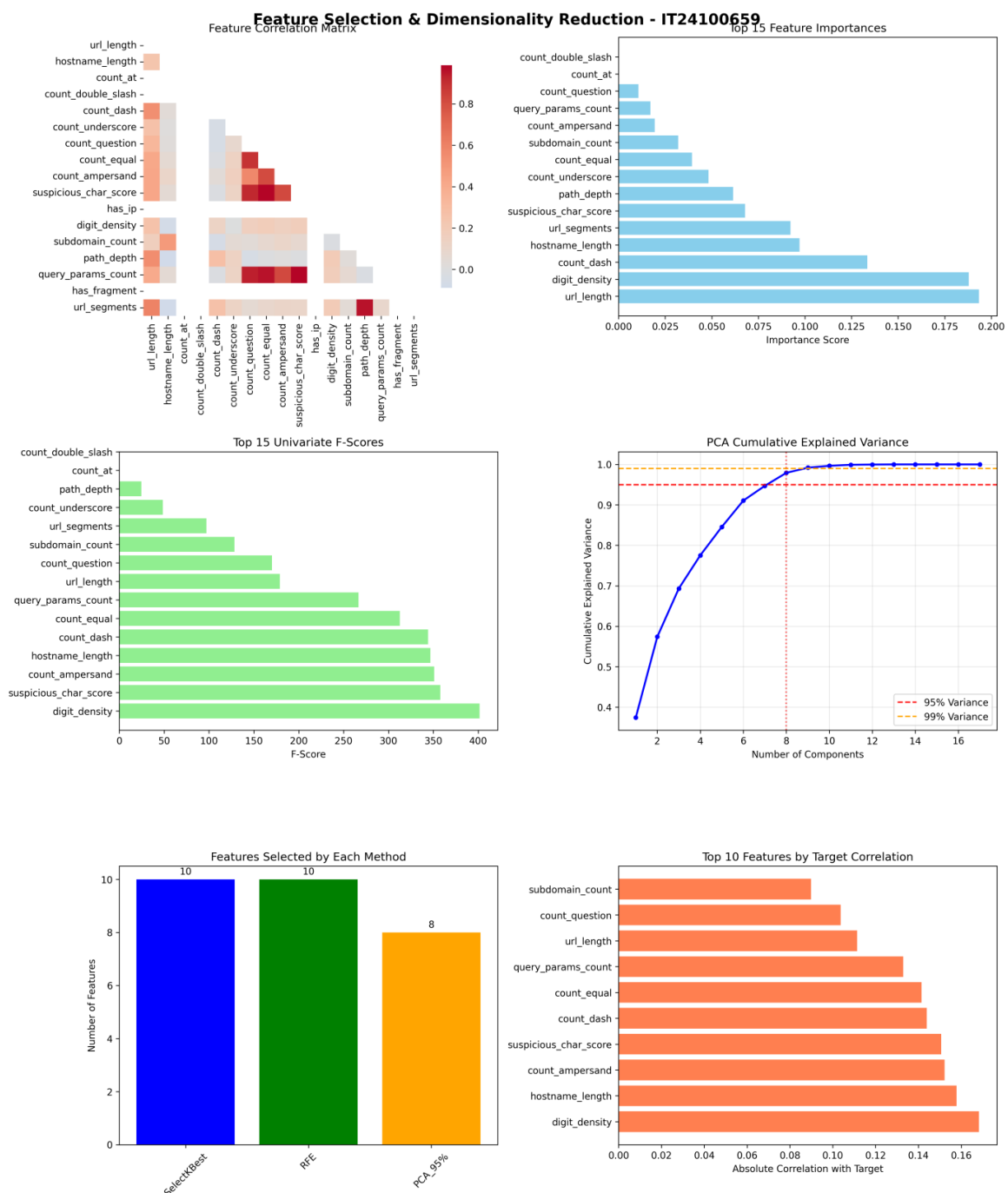
## 3.2) Feature Engineering & Creation Analysis

Feature engineering process involved expanding the feature set, analyzing relationships, and identifying which features contributed most to classification accuracy. Below charts show how the number of features increased after engineering, with a breakdown of feature types like statistical and ratio-based. It also highlights which features were most important and how they relate to each other.



Approach and created signals:

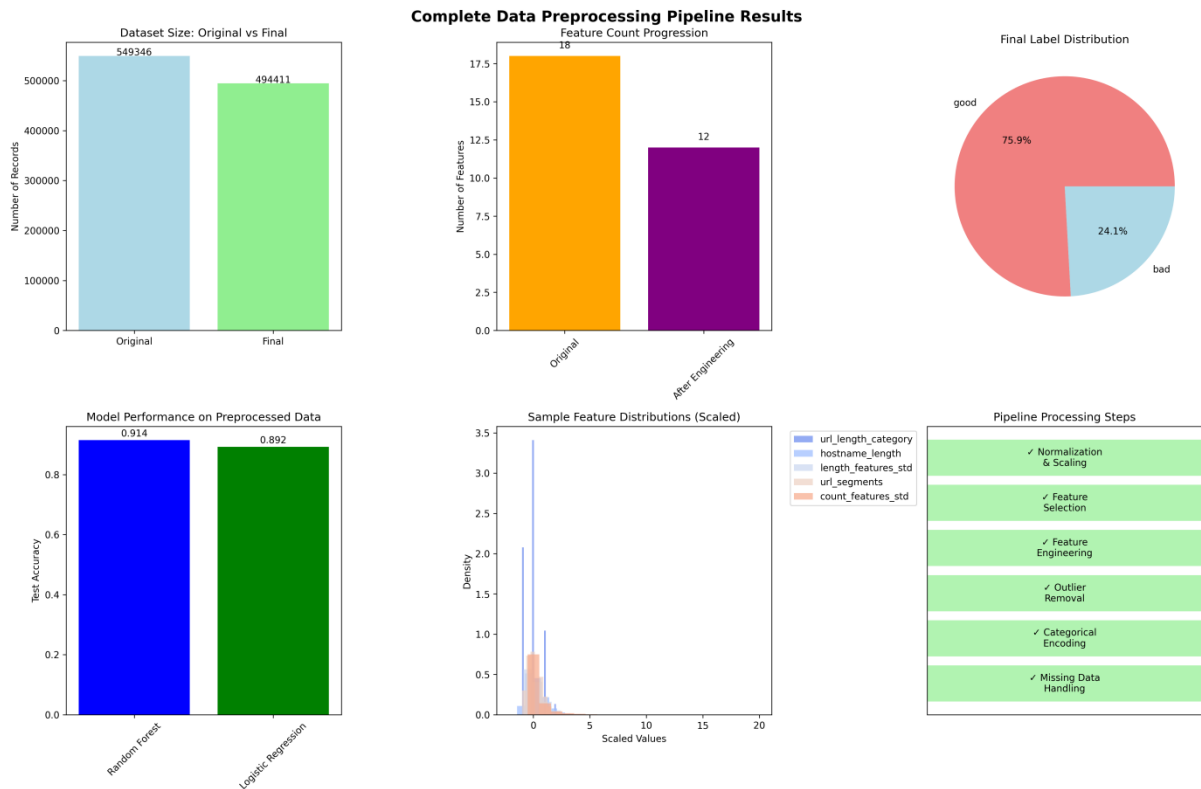
- **Structural features:** hostname\_length, subdomain\_count, url\_segments, path\_depth.
- **Count features (and log transforms):** count\_dash\_log, count\_underscore\_log, count\_features\_std, URL\_encoded\_log.
- **Complexity metrics:** url\_complexity (entropy-like score), length\_features\_std, token-based ratios (url\_hostname\_ratio).
- **Categorical encodings:** url\_length\_category, suspicious\_score categories.
- **Scaling:** MinMaxScaler for tree-insensitive pipelines; Standard/Robust scalers for distance-based models (KNN, Logistic).
- **Feature selection:** removed highly collinear or redundant features after correlation and univariate importance analysis.



### 3.3) Exploratory Data Analysis (EDA)

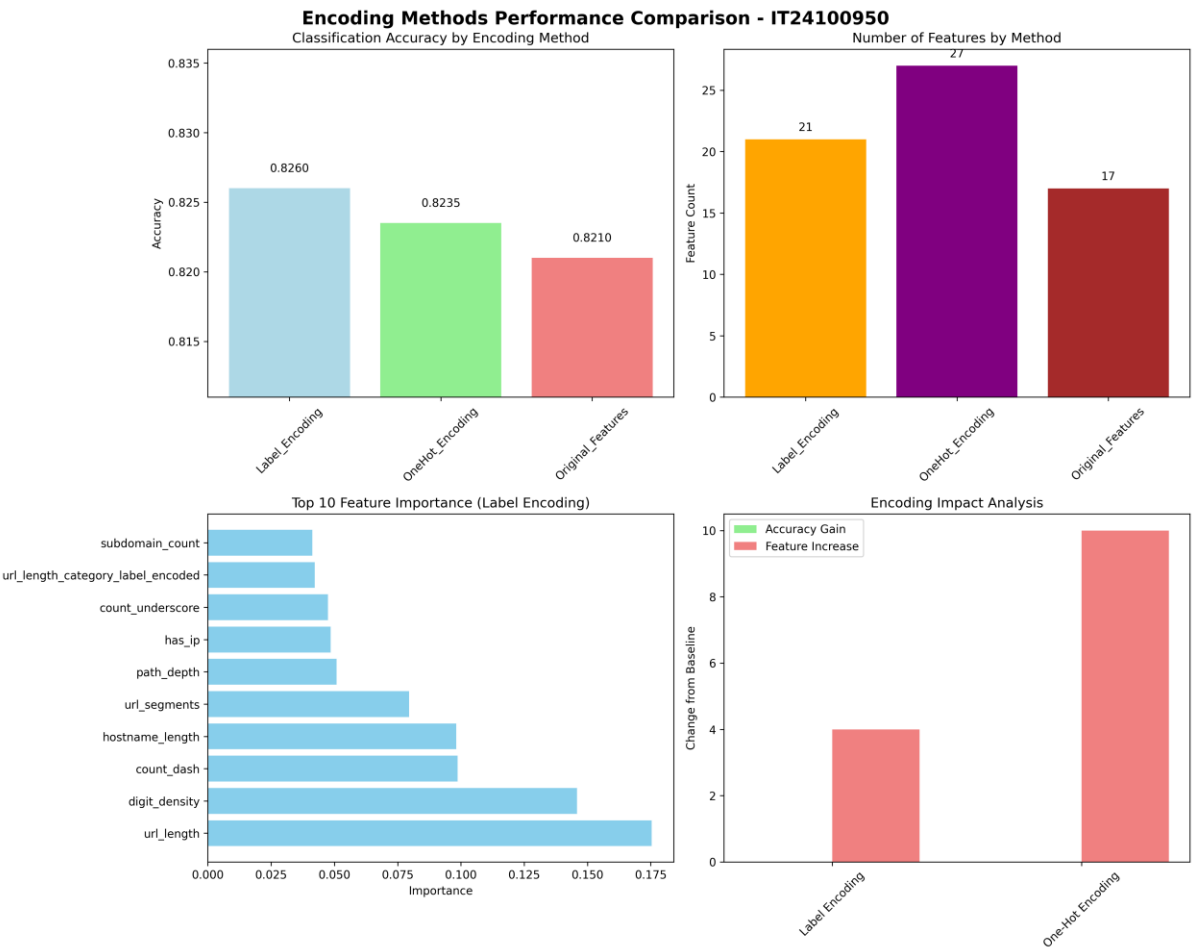
EDA was conducted to understand the dataset's structure, identify potential issues, and guide preprocessing decisions. The following visualizations supported this process:

- **Preprocessing Pipeline Summary:**



This figure outlines the full data journey from raw input to cleaned, scaled, and labeled data. It shows how the dataset size and feature count evolved, and how scaling affected feature distributions.

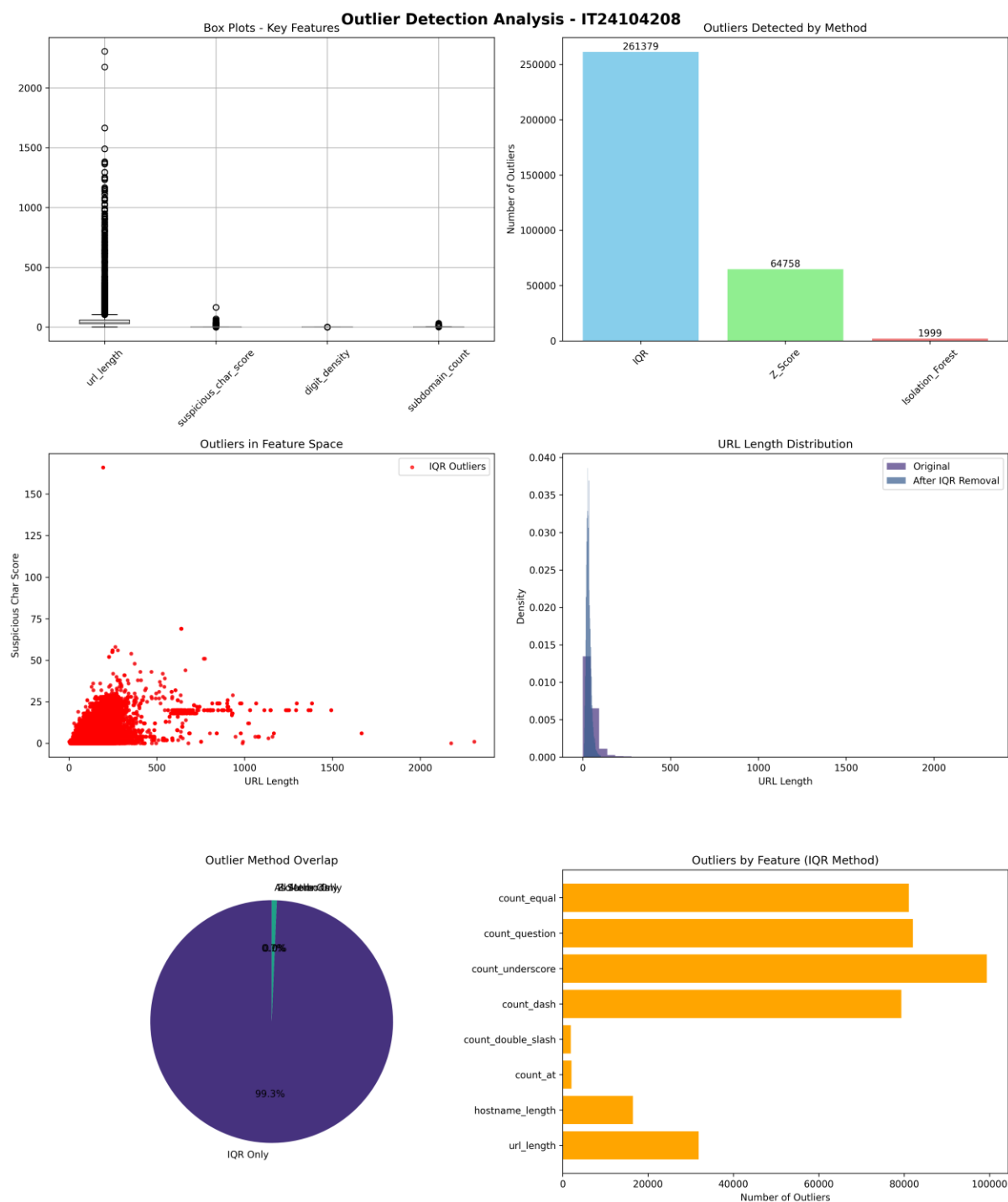
- **Encoding Methods Performance Comparison:**



This chart compares different encoding strategies and their impact on model performance. Label encoding was found to be effective, offering a good balance between accuracy and feature simplicity.

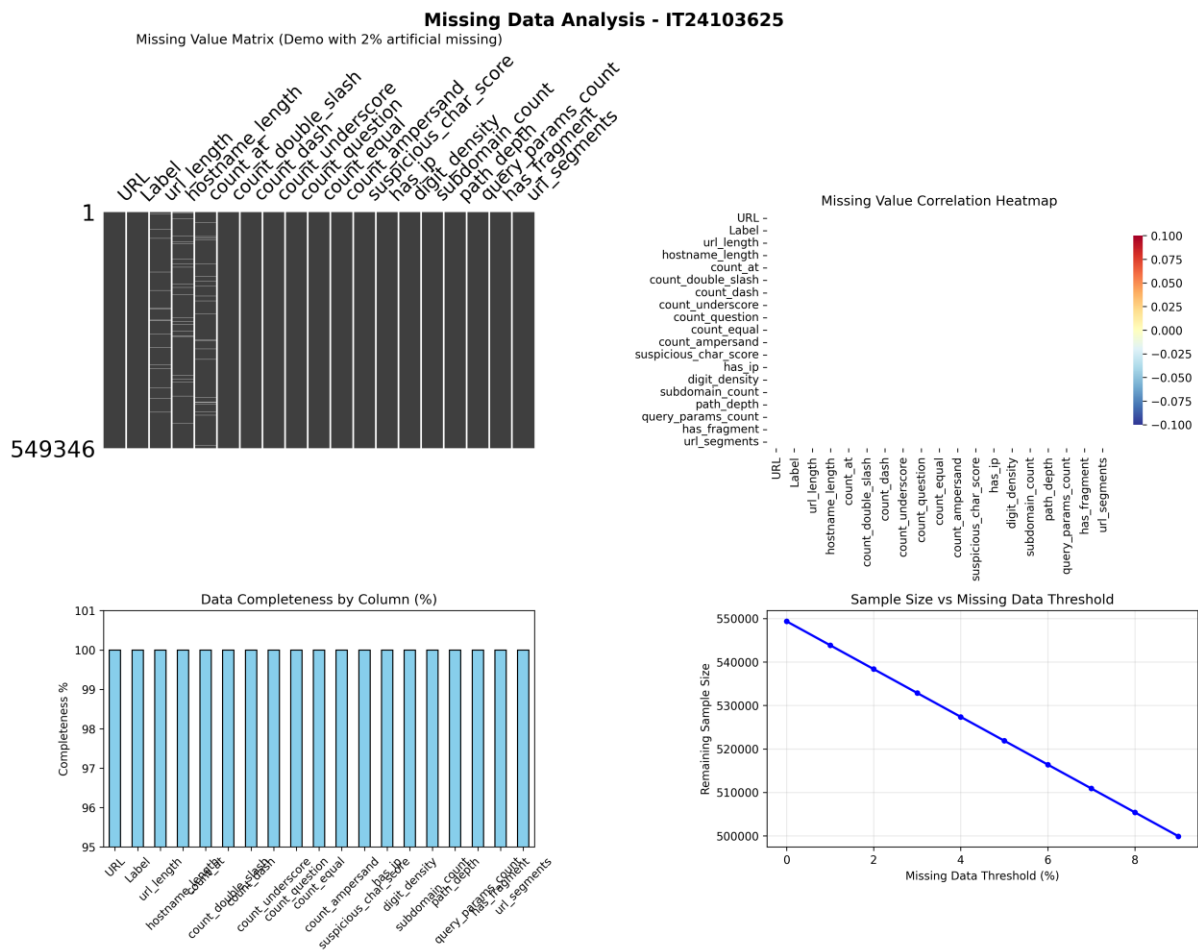


- **Outlier Detection Analysis:**



Outliers were detected using IQR, Isolation Forest, and DBSCAN. Removing these extreme cases helped stabilize model training and reduce noise in feature distributions.

- Missing Data Analysis:



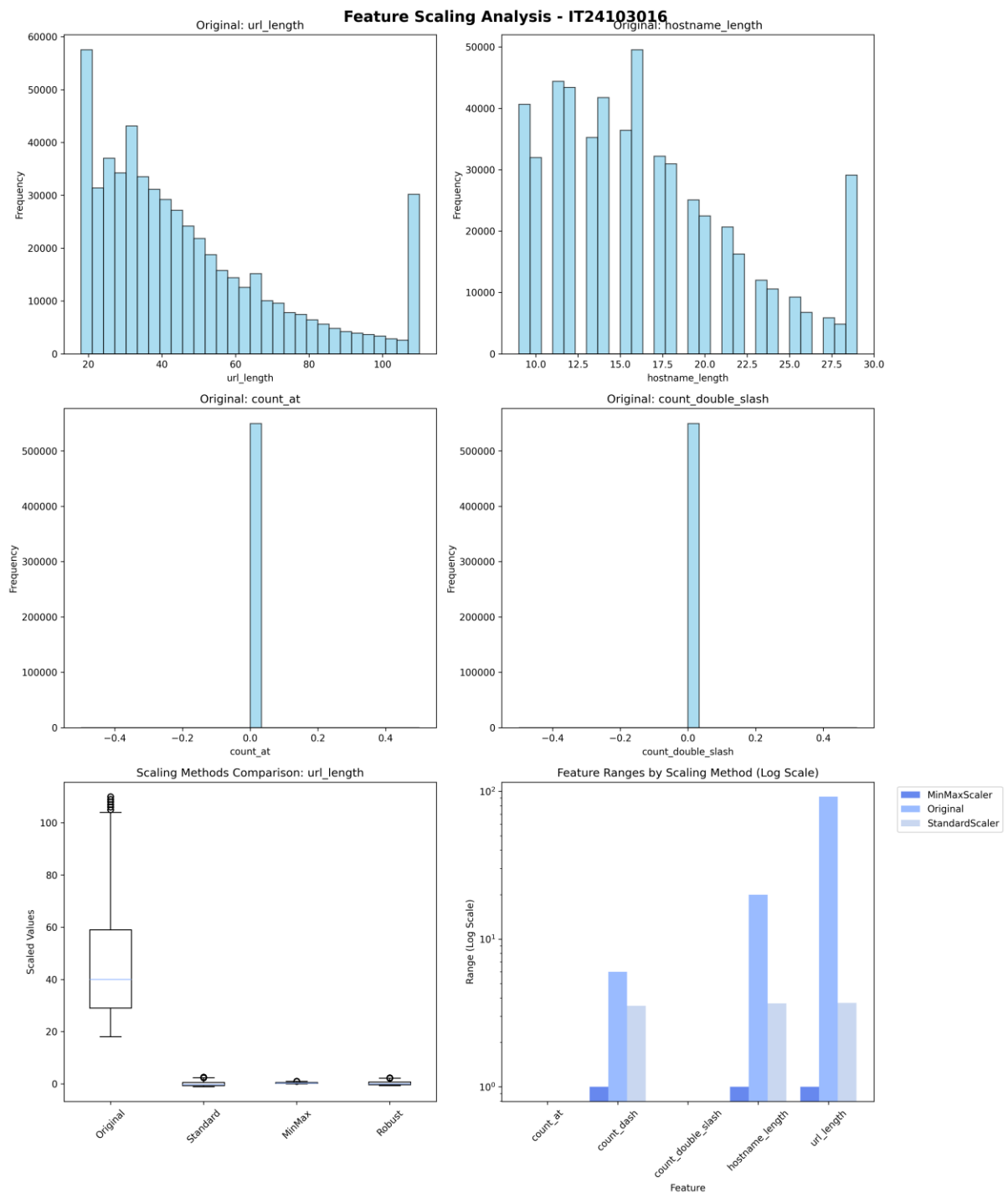
This figure confirms that the dataset had no missing values and shows how the model handled synthetic missingness. The model remained robust even when some features were intentionally masked.

- **Categorical Variables Analysis:**



This visualization explores how categorical features like URL length categories and suspicious scores relate to phishing labels. It helped identify which categories were most predictive.

- **Feature Scaling Analysis:**



This chart compares different scaling methods and their effect on feature distributions. It guided the selection of scalers like StandardScaler and MinMaxScaler for different models.

## 4) Model Design and Implementation

Six classification models with a broad variety of variations were chosen to try out different modeling schemes, ranging from basic baselines to advanced ensembles and probabilistic classifiers. The models included:

- **Logistic Regression:** A linear baseline model for interpretability.
- **Decision Tree:** A simple non-linear model capable of capturing feature interactions.
- **Random Forest:** An ensemble of decision trees that improves generalization.
- **Gradient Boosting:** A sequential ensemble model that corrects previous errors.
- **K-Nearest Neighbors (KNN):** An instance-based model that classifies based on proximity.
- **Naive Bayes:** A probabilistic model based on feature independence assumptions.

### 4.1) Implementation Details

All models were implemented using Python and standard machine learning libraries.

| Model               | Primary Library | Main Class                                  |
|---------------------|-----------------|---------------------------------------------|
| Logistic Regression | scikit-learn    | sklearn.linear_model.LogisticRegression     |
| Decision Tree       | scikit-learn    | sklearn.tree.DecisionTreeClassifier         |
| Random Forest       | scikit-learn    | sklearn.ensemble.RandomForestClassifier     |
| Gradient Boosting   | scikit-learn    | sklearn.ensemble.GradientBoostingClassifier |
| KNN                 | scikit-learn    | sklearn.neighbors.KNeighborsClassifier      |
| Naive Bayes         | scikit-learn    | sklearn.naive_bayes.GaussianNB              |

The features selected for modeling include engineered structural and statistical features. The target variable is Label\_encoded. An 80% training and 20% testing split was consistently applied for all models. Model training used `.fit(X_train, y_train)` and prediction used `.predict(X_test)` functions. Supporting libraries included pandas for data handling, NumPy for numerical operations, and Matplotlib and Seaborn for visualization.

## 4.2) Hyperparameter Tuning

Optimization of model performance included tuning significant hyperparameters. At least three different configurations were trained for each model type to determine the best setup.

Methods:

- **Manual Tuning:** Applied to Logistic Regression by adjusting regularization strength.
- **Grid Search with Cross-Validation (GridSearchCV):** Applied to Decision Tree, Random Forest, Gradient Boosting, KNN, and Naive Bayes. This method rigorously searched over a pre-defined grid of parameter values, with 5-fold cross-validation on the training set to identify the combination that yielded the best average performance.

Key Parameters Tuned:

- **Decision Tree:** max\_depth, min\_samples\_leaf
- **Random Forest:** n\_estimators, max\_depth, min\_samples\_split
- **Gradient Boosting:** learning\_rate, n\_estimators, max\_depth
- **KNN:** n\_neighbors, weights, metric
- **Naive Bayes:** var\_smoothing

This systematic approach ensured that each model was optimized beyond its default parameters before the final comparison.

## 5) Evaluation and Comparison

The evaluation of the performance of each classification model was conducted using a uniform validation system that included quantitative performance measures. To ensure fair evaluation, the dataset was divided into training (80%) and testing (20%) sets using time-based splitting, and the sequence of observations was preserved to avoid information leakage.

The resulting evaluation metrics included:

- **Accuracy:** Overall correctness of predictions.
- **Precision:** True positives among predicted positives.
- **Recall:** True positives among actual positives.
- **F1-score:** Harmonic mean of precision and recall.
- **AUC Score:** Measures the ability to distinguish between classes.
- **Log Loss:** Penalizes incorrect confident predictions.

### 5.1) Model Performance Summary

| Model               | Accuracy | Log Loss | AUC Score | Precision (0/1) | Recall (0/1) | F1-score (0/1) |
|---------------------|----------|----------|-----------|-----------------|--------------|----------------|
| Logistic Regression | 0.8193   | 0.3755   | 0.8958    | 0.89 / 0.77     | 0.72 / 0.91  | 0.80 / 0.83    |
| Decision Tree       | 0.8522   | 2.2472   | -         | 0.86 / 0.84     | 0.84 / 0.87  | 0.85 / 0.85    |
| Random Forest       | 0.8694   | 0.2750   | 0.9492    | 0.90 / 0.84     | 0.83 / 0.91  | 0.86 / 0.87    |
| Gradient Boosting   | 0.8461   | 0.3258   | -         | 0.89 / 0.81     | 0.79 / 0.90  | 0.84 / 0.85    |
| KNN                 | 0.8580   | -        | 0.9405    | 0.89 / 0.83     | 0.82 / 0.90  | 0.85 / 0.86    |
| Naive Bayes         | 0.7705   | 2.0161   | -         | 1.00 / 0.69     | 0.54 / 1.00  | 0.70 / 0.81    |

## 5.2) Analysis of Results

Based on the results above, a strong differentiation of model performances is observable.

- **Logistic Regression** performed moderately well, with an accuracy of 81.9% and an AUC of 0.8958. While it captured linear trends, it struggled with complex patterns.
- **Decision Tree** showed good performance but had a high log loss, indicating overconfident predictions.
- **Random Forest** emerged as the best performer, with the highest AUC (0.9492), lowest log loss (0.2750), and balanced precision and recall.
- **Gradient Boosting** also performed strongly, especially in recall for class 1 (phishing URLs), making it suitable for security-sensitive applications.
- **KNN** showed surprisingly strong results, especially in recall, but may suffer from scalability issues with larger datasets.
- **Naive Bayes** had the lowest accuracy and highest log loss. While its recall for phishing URLs was perfect (1.00), its precision for legitimate URLs was poor, leading to many false positives.

## 5.3) Discussion

Random Forest stands out as the most balanced and reliable model for phishing URL detection. Its ability to capture non-linear relationships and its robustness against overfitting make it ideal for production use. Gradient Boosting and KNN also show promise, especially in recall-heavy scenarios.

Naive Bayes, while fast and simple, may be too aggressive in flagging URLs as phishing, which could lead to user frustration due to false alarms. Logistic Regression and Decision Tree serve as interpretable baselines but fall short in performance compared to ensemble methods.



## 6) Ethical Considerations and Bias Mitigation

The Phishing Site URLs Dataset contains over half a million labeled URLs. While the dataset is structured and clean, ethical considerations remain critical when building and deploying security models. These include risks of bias, overgeneralization, and unintended consequences in real-world applications.

### 6.1) Key Biases can be Happen

I. **Sampling / Population Bias:** The dataset may over represent certain domain types, geographies, or URL structures. This can lead to models that generalize poorly to underrepresented or newer phishing tactics.

II. **Labeling Bias:** Some URLs may be mislabeled or borderline, especially if labeling was automated. This can introduce noise and skew model learning.

III. **Temporal Bias:** The dataset may not reflect current phishing trends. Attackers evolve quickly, and models trained on older data may underperform on new threats.

IV. **Feature Bias:** Certain structural features (e.g., long subdomains, encoded characters) may correlate with legitimate services used by specific groups, leading to disproportionate false positives.

V. **Linkage Risk:** URLs, especially when combined with timestamps or user-agent data, can be used to re-identify individuals or infer browsing behavior.

### 6.2) Biases Mitigation strategies

#### 6.2.1) *Data handling & privacy*

To protect user privacy and reduce linkage risk:

- Avoid storing or sharing raw URLs in production systems.
- Use hashed or tokenized representations when logging or analyzing.
- Share only aggregated statistics (e.g., counts by domain or feature) when publishing results.

### ***6.2.2) Bias Detection and data Quality***

To detect and mitigate bias in the dataset:

- Perform exploratory bias checks by analyzing feature distributions across labels.
- Use stratified sampling or reweighting to ensure balanced representation during training.
- Monitor model performance across different URL categories (e.g., TLDs, length buckets).
- Use time-based validation (train on earlier data, test on later) to detect temporal drift.

### ***6.2.3) Modeling & fairness***

To ensure fairness and transparency in model predictions:

- Define fairness metrics relevant to the use case (e.g., equal false positive rates across domains).
- Use interpretable models or add explainability layers (e.g., SHAP) to justify decisions.
- Avoid using the model for high-stakes decisions (e.g., blocking access) without human review.

### ***6.2.4) Documentation & governance***

To promote transparency and accountability:

- Publish a model card and dataset datasheet outlining provenance, time range, known limitations, and intended uses.
- Restrict access to raw data through role-based permissions and data-use agreements.
- Schedule periodic audits and red-team evaluations to assess fairness and robustness.

## 6.3) Ethical considerations & recommendations

- **Consent & Purpose:** Even public datasets carry ethical obligations. Ensure analyses align with the dataset's license and the reasonable expectations of data subjects.
- **Minimize Harm:** Avoid publishing outputs that could be used to target individuals or domains unfairly.
- **Transparency & Accountability:** Clearly state model limitations and document who is responsible for decisions made using the model.
- **Responsible Reporting:** When sharing findings, include bias analyses, demographic coverage, and potential misuse scenarios.
- **Continual Review:** If deployed, schedule regular re-evaluation of the model to detect drift and emerging biases.

## 7) Reflections and Lessons Learned

The main goal of this project was to build a reliable phishing detection system using URL structure alone. Throughout the project, we gained hands-on experience in implementing a complete machine learning pipeline, including data preprocessing, exploratory data analysis (EDA), feature engineering, model design, and evaluation.

From a technical perspective, we learned:

- The importance of feature engineering in structured data problems.
- How to handle class imbalance and outliers effectively.
- The trade-offs between model complexity, interpretability, and performance.
- How hyperparameter tuning and cross-validation improve generalization.

From a collaborative perspective, we improved our:

- Team coordination and task delegation.
- Communication of technical findings through visualizations and documentation.
- Awareness of ethical responsibilities in AI development.

Looking forward, we aim to:

- Explore advanced bias detection and explainability techniques.
- Integrate real-time URL scanning and feedback loops.
- Deploy the best-performing model as a browser plugin or API.
- Continuously monitor model performance and fairness in production.

This project deepened our understanding of how to design machine learning systems that are both accurate and responsible.

## 8) References

- 1) *Google Colab*. (n.d.). Google.com. Retrieved October 20, 2025, from <https://colab.research.google.com/>
- 2) *Matplotlib - Visualization with Python*. (n.d.). Matplotlib.org. Retrieved October 20, 2025, from <https://matplotlib.org/>
- 3) *NumPy*. (n.d.). Numpy.org. Retrieved October 20, 2025, from <https://numpy.org/>
- 4) *pandas*. (n.d.). Pydata.org. Retrieved October 20, 2025, from <https://pandas.pydata.org/>
- 5) *Scikit-learn*. (n.d.). Scikit-learn.org. Retrieved October 20, 2025, from <https://scikit-learn.org/>
- 6) *Seaborn: Statistical Data Visualization*. (n.d.). Pydata.org. Retrieved October 20, 2025, from <https://seaborn.pydata.org/>
- 7) *XGBoost*. (n.d.). Xgboost.ai. Retrieved October 20, 2025, from <https://xgboost.ai/>
- 8) *UCI Machine Learning Repository*. (n.d.). Uci.edu. Retrieved October 20, 2025, from <https://archive.ics.uci.edu/dataset/502/online+retail+ii>